IE-494 Lab 10 — Working with DynamoDB

Course: IE-494 Big Data Processing · Autumn 2026
Instructor: PM Jat (pm_jat@dau.ac.in)
Student: Aditya Vaish · ID 202401401
Submitted: 2 November 2026
Stack: DynamoDB Local 3.3.2, AWS SDK for Java 2.31.0 (dynamodb, dynamodb-enhanced), Java (release 17), Maven, macOS arm64

1. Assignment

Take CUSTOMER, ORDERS and LINEITEM from the TPC-H dataset (scale factor 1, generated with dbgen as in Lab 08: 150,000 customers, 1,500,000 orders, 6,001,215 line items), design a DynamoDB database for the query load below, populate it, and answer every query in a Java program with one function per query, using the high-level (Enhanced Client) API.

1. Retrieve a customer's profile.
2. Retrieve all orders for a customer.
3. Retrieve all line items belonging to an order.
4. Retrieve the details required to display a customer's order history.
5. List all items ordered by a customer in a year.
6. List all items a customer ordered in the previous year (year-1) but has not bought this year.
7. Show the total value of a customer's orders in a given year.

2. Database design

Every query is driven by one customer (1, 2, 4, 5, 6, 7) or one order (3), so the data is modelled as **aggregates** in a **single table `TPCH`** with a composite primary key pk (partition, string) and sk (sort, string):

- A customer's partition CUST#<custkey> holds the profile item (sk = PROFILE) and one item per order.
- An order item's sort key ORDER#<orderdate>#<orderkey> starts with the date, so the orders of a customer are stored in date order and "orders of year Y" is just begins_with(sk, "ORDER#Y"). The 10-digit zero-padded order key at the end keeps the key unique.
- The line items of an order are **embedded** in the order item as a list of maps (the CORDERS aggregate of Lab 08). TPC-H orders have 1–7 line items, so an item is at most a few KB, far below the 400 KB limit, and every order-level query gets the line items for free.
- Query 3 is addressed by order key only, so a **global secondary index `OrderIndex`** with partition key orderkey (projection KEYS_ONLY) maps an order key to the order's (pk, sk); a GetItem then returns the order with its line items. KEYS_ONLY keeps the index small: it stores only pk, sk and orderkey per order.
- No Scan is needed for any of the seven queries; every query is a key lookup or a single-partition Query.

item	pk	sk	other attributes
customer profile	CUST#<custkey>	PROFILE	type, custkey, name, address, nationkey, phone, acctbal, mktsegment, comment
order with its line items	CUST#<custkey>	ORDER#<orderdate>#<orderkey, 10 digits>	type, orderkey, custkey, orderstatus, totalprice, orderdate, year, orderpriority, clerk, shippriority, comment, lineitems = list of maps {linenumber, partkey, suppkey, quantity, extendedprice, discount, tax, returnflag, linestatus, shipdate, commitdate, receiptdate, shipinstruct, shipmode, comment}

Items as stored in DynamoDB:

```
{ "pk": "CUST#121", "sk": "PROFILE", "type": "CUSTOMER", "custkey": 121, "name": "Customer#000000121", "address": "...",
  "nationkey": 17, "phone": "27-411-990-2959", "acctbal": 6428.32, "mktsegment": "BUILDING", "comment": "..." }

{ "pk": "CUST#121", "sk": "ORDER#1994-06-26#0005512869", "type": "ORDER", "orderkey": 5512869, "custkey": 121,
  "orderstatus": "F", "totalprice": 133146.02, "orderdate": "1994-06-26", "year": 1994, "orderpriority": "...", "clerk": "Clerk#...",
  "shippriority": 0, "comment": "...",
  "lineitems": [ { "linenumber": 1, "partkey": ..., "suppkey": ..., "quantity": ..., "extendedprice": ..., "discount": ..., "tax": ...,
                   "returnflag": "...", "linestatus": "...", "shipdate": "...", "commitdate": "...", "receiptdate": "...",
                   "shipinstruct": "...", "shipmode": "...", "comment": "..." }, ... ] }
```

Query load → DynamoDB operations

#	query	operation	requests
1	Retrieve a customer's profile	GetItem (CUST#c, PROFILE)	1
2	Retrieve all orders for a customer	Query pk = CUST#c, sk begins_with ORDER#	1 (paged)
3	Retrieve all line items of an order	Query GSI OrderIndex orderkey = o → (pk, sk); GetItem the order; line items are inside it	2
4	Customer's order history	GetItem profile + Query pk = CUST#c, sk begins_with ORDER# (every order already carries its line items)	2
5	Items ordered by a customer in a year Y	Query pk = CUST#c, sk begins_with ORDER#Y; collect the embedded line items	1

#	query	operation	requests
6	Items ordered in Y-1 but not in Y	`Query` for ORDER#(Y-1) and `Query` for ORDER#Y; set difference of partkeys in the program	2
7	Total value of a customer's orders in year Y	`Query` pk = CUST#c, sk begins_with ORDER#Y; sum O_TOTALPRICE	1

3. Setup

- **DynamoDB Local** (`dynamodb_local_latest`, version 3.3.2) started as in the setup sheet: `java -Djava.library.path=./DynamoDBLocal_lib -jar DynamoDBLocal.jar -sharedDb -port 8000 -dbPath lab10/db`. The program connects with endpoint `http://localhost:8000`, region `ap-south-1`, access key / secret `1234` / `1234`.
- **Data:** `customer.tbl`, `orders.tbl`, `lineitem.tbl` from `dbgen -s 1` (the 1 GB TPC-H population used in Labs 08 and 09).
- **Dependencies** (from the lab sheet): AWS SDK BOM 2.31.0, `dynamodb`, `dynamodb-enhanced`, `slf4j-simple` 1.7.36.
- **Run:** `bash lab10/run_all.sh` → create table, load, describe, then the seven queries for customer 121, its order 5512869 and year 1994. Customer 121 was chosen because it has 15 orders with orders in two consecutive years (3 in 1993, 5 in 1994), which exercises queries 5–7.

4. How it works

- **Model classes** (`Customer`, `Order`, `LineItem`): `@DynamoDbBean` POJOs. `pk`/`sk` carry `@DynamoDbPartitionKey`/`@DynamoDbSortKey`; `Order.orderkey` carries `@DynamoDbSecondaryPartitionKey(indexNames = "OrderIndex")`; `Order.lineitems` is a `List<LineItem>`, which the Enhanced Client maps to a DynamoDB list of maps. Money columns are `BigDecimal` (DynamoDB numbers).
- **Create** (`Schema.create`): `DynamoDbTable<Order>.createTable(...)` with an `EnhancedGlobalSecondaryIndex` for `OrderIndex`; the key schema and attribute definitions come from the bean annotations. On-demand (PAY_PER_REQUEST) billing.
- **Load** (`TpchLoader`): `customer.tbl` rows become profile items. `orders.tbl` and `lineitem.tbl` are both written by dbgen in ascending order key, so they are merge-joined in one streaming pass: read an order, take its line items from the head of `lineitem.tbl`, put the list into the order bean. Items go out as `BatchWriteItem` requests of 25 (`DynamoDbEnhancedClient.batchWriteItem` with a `WriteBatch` per request), 8 writer threads, unprocessed items re-sent with backoff.
- **Queries** (`Queries`, one method each): `DynamoDbTable.getItem(Key)`, `DynamoDbTable.query(QueryConditional.sortBeginsWith(...))` and `DynamoDbIndex.query(QueryConditional.keyEqualTo(...))`. Query results are iterated page by page; the program counts the pages and the elapsed time. Query 6 is two Queries plus a set difference of part keys in Java; Query 7 sums `totalprice` and cross-checks it against Σ extendedprice × (1 − discount) × (1 + tax) of the embedded line items.

5. Results

	value
customer.tbl rows → profile items	150,000
orders.tbl rows → order items	1,500,000
lineitem.tbl rows (embedded in the orders)	6,001,215
profile items put	150,000 in 66.9 s (2,242 items/s)
order items put	1,500,000 in 1901.6 s (789 items/s)
items in the table	1,650,000
batch retries	0
describe-table item count / size	1,650,000 / 1,881,362,746 bytes (1794.2 MiB)

Requests and wall time of each query (customer 121, order 5512869, year 1994), from `output/04_queries.txt`:

query	requests	time
Q1: retrieve the profile of customer 121	1 GetItem	98.1 ms
Q2: all orders of customer 121	1 Query page(s)	44.4 ms
Q3: all line items of order 5512869	1 index Query page(s) + 1 GetItem	11.4 ms
Q4: order history of customer 121 (profile + each order with its line items)	1 GetItem + 1 Query page(s)	18.3 ms
Q5: items ordered by customer 121 in 1994	1 Query page(s)	11.7 ms
Q6: items customer 121 ordered in 1993 but not in 1994	2 Query page(s)	16.1 ms
Q7: total value of the orders of customer 121 in 1994	1 Query page(s)	13.3 ms

- **Q1** is a single key lookup. **Q2, Q5, Q7** are one single-partition Query each, and the year queries read only the orders of that year because the year is a prefix of the sort key. **Q4** needs one extra GetItem for the profile.
- **Q3** costs one index Query plus one GetItem, the price of a KEYS_ONLY projection; projecting ALL would make it one request at the cost of duplicating every order (with its line items) into the index.
- **Q6** cannot be a single DynamoDB request (set difference is not a key condition); it is two year Queries and a `TreeSet` difference in the client. TPC-H part keys are random over 200,000 parts, so the two years share no part and the answer is every part of year-1.
- **Q7:** the sum of `totalprice` (498,215.67) and the value recomputed from the embedded line items (498,215.88) differ by 21 cents because dbgen truncates each line's extendedprice × (1 − discount) × (1 + tax) to whole cents before adding it into O_TOTALPRICE; both numbers

come from the same single Query.

- Load throughput is bounded by DynamoDB Local (one SQLite database behind the HTTP endpoint), not by the client; profile items are small and order items carry 1–7 line items each.

6. Program outputs

6.1 Create table (`output/01_create.txt`)

```
creating table TPCH (pk S, sk S) with GSI OrderIndex (orderkey N, KEYS_ONLY), on-demand capacity ...
created.

======== describe-table TPCH ========
status               : ACTIVE
billing              : PAY_PER_REQUEST
key schema           : pk (HASH), sk (RANGE)
attribute definitions: orderkey N  sk S  pk S
GSI OrderIndex        : orderkey (HASH), projection KEYS_ONLY, status ACTIVE
item count           : 0
table size           : 0 bytes (0.0 MiB)
```

6.2 Load (`output/02_load.txt`)

```
customer.tbl = /Users/avaish/IE-494/lab8/data/customer.tbl (24196144 bytes)
orders.tbl   = /Users/avaish/IE-494/lab8/data/orders.tbl (170452161 bytes)
lineitem.tbl = /Users/avaish/IE-494/lab8/data/lineitem.tbl (753862072 bytes)

---- phase 1: CUSTOMER -> profile items (pk=CUST#custkey, sk=PROFILE) ----
  CUSTOMER submitted    100,000  written     99,200     42.3 s    2,344 items/s
  CUSTOMER submitted    150,000  written    150,000     66.9 s    2,242 items/s

---- phase 2: ORDERS + LINEITEM -> order items (pk=CUST#custkey, sk=ORDER#date#orderkey) ----
  ORDER    submitted    100,000  written     99,200     65.8 s    1,507 items/s
  ORDER    submitted    200,000  written    199,200    159.5 s    1,249 items/s
  ORDER    submitted    300,000  written    299,200    276.4 s    1,082 items/s
  ORDER    submitted    400,000  written    399,200    358.6 s    1,113 items/s
  ORDER    submitted    500,000  written    499,200    432.8 s    1,154 items/s
  ORDER    submitted    600,000  written    599,200    560.7 s    1,069 items/s
  ORDER    submitted    700,000  written    699,200    753.0 s      929 items/s
  ORDER    submitted    800,000  written    799,200    908.1 s      880 items/s
  ORDER    submitted    900,000  written    899,200   1076.3 s      835 items/s
  ORDER    submitted  1,000,000  written    999,200   1234.4 s      809 items/s
  ORDER    submitted  1,100,000  written  1,099,200   1400.8 s      785 items/s
  ORDER    submitted  1,200,000  written  1,199,200   1558.7 s      769 items/s
  ORDER    submitted  1,300,000  written  1,299,200   1665.0 s      780 items/s
  ORDER    submitted  1,400,000  written  1,399,200   1744.5 s      802 items/s
  ORDER    submitted  1,500,000  written  1,499,200   1899.9 s      789 items/s
  ORDER    submitted  1,500,000  written  1,500,000   1901.6 s      789 items/s

======== load summary ========
customer.tbl rows   : 150,000
orders.tbl rows     : 1,500,000
lineitem.tbl rows   : 6,001,215 (embedded in their orders)
profile items put   : 150,000 in 66.9 s (2,242 items/s)
order items put     : 1,500,000 in 1901.6 s (789 items/s)
items in table      : 1,650,000
batch retries       : 0 (unprocessed items re-sent)
writer threads      : 8, 25 items per BatchWriteItem
```

6.3 describe-table after the load (`output/03_describe.txt`)

```
======== describe-table TPCH ========
status               : ACTIVE
billing              : PAY_PER_REQUEST
key schema           : pk (HASH), sk (RANGE)
attribute definitions: orderkey N  sk S  pk S
GSI OrderIndex        : orderkey (HASH), projection KEYS_ONLY, status ACTIVE
item count           : 1,650,000
table size           : 1,881,362,746 bytes (1794.2 MiB)
```

6.4 Q1: retrieve the profile of customer 121

```
GetItem  pk=CUST#121  sk=PROFILE
custkey    : 121
name       : Customer#000000121
address    : tv nCR2YKupGN73mQudO
nationkey  : 17
phone      : 27-411-990-2959
acctbal    : 6428.32
mktsegment : BUILDING
comment    : uriously stealthy ideas. carefully final courts use carefully
(1 GetItem, 98.1 ms)
```

6.5 Q2: all orders of customer 121

```
Query  pk=CUST#121  sk begins_with ORDER#
orderdate  orderkey   status priority        clerk           totalprice lines
1992-03-31 3177383    F      5-LOW           Clerk#000000789    188986.83     4
1992-10-21 3279495    F      1-URGENT        Clerk#000000078    102928.39     5
1993-06-24 3568994    F      5-LOW           Clerk#000000294    210567.55     4
1993-10-16 2970276    F      1-URGENT        Clerk#000000270    110090.35     2
1993-12-26 2383682    F      3-MEDIUM        Clerk#000000533      3087.81     1
1994-01-24 3960641    F      5-LOW           Clerk#000000376     77312.55     2
1994-04-09 2174404    F      3-MEDIUM        Clerk#000000397     55650.77     3
1994-06-26 5512869    F      4-NOT SPECIFIED Clerk#000000408    133146.02     7
1994-10-01 5299460    F      2-HIGH          Clerk#000000570    147099.69     5
```

```
1994-10-10 202628     F      4-NOT SPECIFIED Clerk#000000179     85006.64     3
1995-08-06 3797444    O      4-NOT SPECIFIED Clerk#000000779    210230.11     5
1996-06-19 4240038    O      3-MEDIUM        Clerk#000000055    124670.76     2
1997-09-02 464128     O      1-URGENT        Clerk#000000604    215871.48     6
1997-12-31 5783393    O      4-NOT SPECIFIED Clerk#000000779     93382.65     4
1998-07-27 1432258    O      1-URGENT        Clerk#000000590     77070.43     4
15 orders
(1 Query page(s), 44.4 ms)
```

6.6 Q3: all line items of order 5512869

```
Query  index OrderIndex  orderkey=5512869
  -> pk=CUST#121  sk=ORDER#1994-06-26#0005512869
GetItem  pk=CUST#121  sk=ORDER#1994-06-26#0005512869
orderdate  orderkey   status priority        clerk            totalprice lines
1994-06-26 5512869    F      4-NOT SPECIFIED Clerk#000000408    133146.02     7
line partkey  suppkey   qty     extprice  disc   tax rf ls shipdate   commitdate receipt    shipinstruct      mode
1    158243   5789        1      1301.24  0.02  0.04 A  F  1994-07-23 1994-08-08 1994-07-26 DELIVER IN PERSON SHIP
2    173959   3960        1      2032.95  0.02  0.03 A  F  1994-07-05 1994-08-02 1994-07-06 DELIVER IN PERSON FOB
3    123950   8975       40     78958.00  0.07  0.04 R  F  1994-09-01 1994-08-26 1994-09-27 TAKE BACK RETURN  REG AIR
4    26304    6305        1      1230.30  0.05  0.06 A  F  1994-10-02 1994-09-23 1994-10-06 TAKE BACK RETURN  SHIP
5    83046    5555       31     31900.24  0.09  0.00 R  F  1994-09-10 1994-09-12 1994-09-14 TAKE BACK RETURN  FOB
6    149742   9743        4      7166.96  0.00  0.03 R  F  1994-08-06 1994-09-10 1994-08-26 COLLECT COD       AIR
7    100340   5361       12     16084.08  0.04  0.02 R  F  1994-07-20 1994-09-09 1994-08-18 DELIVER IN PERSON REG AIR
7 line items
(1 index Query page(s) + 1 GetItem, 11.4 ms)
```

6.7 Q4: order history of customer 121 (profile + each order with its line items)

```
GetItem  pk=CUST#121  sk=PROFILE
Query    pk=CUST#121  sk begins_with ORDER#
Customer#000000121  (custkey 121, BUILDING, nation 17, phone 27-411-990-2959, balance 6428.32)

ORDER 3177383  1992-03-31  status F  5-LOW  clerk Clerk#000000789  total 188986.83
    line 1  part 30291   supp 292   qty  34  ext   41523.86  disc 0.10  tax 0.00  RF  ship 1992-07-06 REG AIR
    line 2  part 89411   supp 1920  qty  30  ext   42012.30  disc 0.03  tax 0.00  RF  ship 1992-07-07 FOB
    line 3  part 128698  supp 3723  qty  44  ext   75974.36  disc 0.04  tax 0.02  AF  ship 1992-06-09 MAIL
    line 4  part 14288   supp 6790  qty  31  ext   37270.68  disc 0.05  tax 0.03  AF  ship 1992-07-15 FOB

ORDER 3279495  1992-10-21  status F  1-URGENT  clerk Clerk#000000078  total 102928.39
    line 1  part 4074    supp 6575  qty  22  ext   21517.54  disc 0.09  tax 0.08  RF  ship 1992-12-29 TRUCK
    line 2  part 55553   supp 5554  qty  11  ext   16594.05  disc 0.06  tax 0.00  RF  ship 1992-12-29 RAIL
    line 3  part 62881   supp 5388  qty   4  ext    7375.52  disc 0.07  tax 0.08  RF  ship 1993-01-07 MAIL
    line 4  part 35764   supp 771   qty  22  ext   37394.72  disc 0.06  tax 0.08  AF  ship 1992-11-25 TRUCK
    line 5  part 85126   supp 5127  qty  19  ext   21111.28  disc 0.07  tax 0.06  AF  ship 1993-01-04 SHIP

ORDER 3568994  1993-06-24  status F  5-LOW  clerk Clerk#000000294  total 210567.55
    line 1  part 175816  supp 5817  qty  49  ext   92698.69  disc 0.04  tax 0.02  AF  ship 1993-08-31 SHIP
    line 2  part 123613  supp 1150  qty   6  ext    9819.66  disc 0.04  tax 0.07  AF  ship 1993-06-27 MAIL
    line 3  part 161673  supp 4190  qty  34  ext   58978.78  disc 0.02  tax 0.06  RF  ship 1993-10-15 AIR
    line 4  part 36749   supp 1756  qty  31  ext   52257.94  disc 0.10  tax 0.03  AF  ship 1993-09-03 FOB

ORDER 2970276  1993-10-16  status F  1-URGENT  clerk Clerk#000000270  total 110090.35
    line 1  part 42471   supp 2472  qty  47  ext   66433.09  disc 0.09  tax 0.07  RF  ship 1994-01-19 MAIL
    line 2  part 196580  supp 1619  qty  28  ext   46944.24  disc 0.07  tax 0.04  AF  ship 1993-10-27 MAIL

ORDER 2383682  1993-12-26  status F  3-MEDIUM  clerk Clerk#000000533  total 3087.81
    line 1  part 195405  supp 5406  qty   2  ext    3000.80  disc 0.02  tax 0.05  AF  ship 1994-02-09 MAIL

ORDER 3960641  1994-01-24  status F  5-LOW  clerk Clerk#000000376  total 77312.55
    line 1  part 162920  supp 7953  qty   2  ext    3965.84  disc 0.05  tax 0.06  AF  ship 1994-04-10 FOB
    line 2  part 188887  supp 1406  qty  38  ext   75083.44  disc 0.07  tax 0.05  AF  ship 1994-04-18 SHIP

ORDER 2174404  1994-04-09  status F  3-MEDIUM  clerk Clerk#000000397  total 55650.77
    line 1  part 146032  supp 3575  qty  13  ext   14014.39  disc 0.10  tax 0.01  AF  ship 1994-07-28 TRUCK
    line 2  part 148863  supp 8864  qty  15  ext   28677.90  disc 0.08  tax 0.03  AF  ship 1994-04-12 FOB
    line 3  part 138205  supp 719   qty  13  ext   16161.60  disc 0.09  tax 0.07  RF  ship 1994-07-21 AIR

ORDER 5512869  1994-06-26  status F  4-NOT SPECIFIED  clerk Clerk#000000408  total 133146.02
    line 1  part 158243  supp 5789  qty   1  ext    1301.24  disc 0.02  tax 0.04  AF  ship 1994-07-23 SHIP
    line 2  part 173959  supp 3960  qty   1  ext    2032.95  disc 0.02  tax 0.03  AF  ship 1994-07-05 FOB
    line 3  part 123950  supp 8975  qty  40  ext   78958.00  disc 0.07  tax 0.04  RF  ship 1994-09-01 REG AIR
    line 4  part 26304   supp 6305  qty   1  ext    1230.30  disc 0.05  tax 0.06  AF  ship 1994-10-02 SHIP
    line 5  part 83046   supp 5555  qty  31  ext   31900.24  disc 0.09  tax 0.00  RF  ship 1994-09-10 FOB
    line 6  part 149742  supp 9743  qty   4  ext    7166.96  disc 0.00  tax 0.03  RF  ship 1994-08-06 AIR
    line 7  part 100340  supp 5361  qty  12  ext   16084.08  disc 0.04  tax 0.02  RF  ship 1994-07-20 REG AIR

ORDER 5299460  1994-10-01  status F  2-HIGH  clerk Clerk#000000570  total 147099.69
    line 1  part 120498  supp 3011  qty  17  ext   25814.33  disc 0.02  tax 0.06  AF  ship 1994-10-25 AIR
    line 2  part 4077    supp 9078  qty   3  ext    2943.21  disc 0.07  tax 0.01  AF  ship 1994-11-26 RAIL
    line 3  part 114001  supp 1535  qty  49  ext   49735.00  disc 0.03  tax 0.02  AF  ship 1994-11-03 FOB
    line 4  part 13391   supp 8394  qty  22  ext   28696.58  disc 0.03  tax 0.03  RF  ship 1994-10-02 TRUCK
    line 5  part 47432   supp 9937  qty  30  ext   41382.90  disc 0.07  tax 0.03  AF  ship 1994-11-30 TRUCK

ORDER 202628  1994-10-10  status F  4-NOT SPECIFIED  clerk Clerk#000000179  total 85006.64
    line 1  part 7049    supp 7050  qty  22  ext   21032.88  disc 0.07  tax 0.05  AF  ship 1994-10-13 SHIP
    line 2  part 59993   supp 5004  qty  16  ext   31247.84  disc 0.00  tax 0.04  RF  ship 1994-11-05 MAIL
    line 3  part 92560   supp 5070  qty  20  ext   31051.20  disc 0.01  tax 0.04  RF  ship 1994-11-29 FOB

ORDER 3797444  1995-08-06  status O  4-NOT SPECIFIED  clerk Clerk#000000779  total 210230.11
    line 1  part 180118  supp 7673  qty   2  ext    2396.22  disc 0.05  tax 0.03  NO  ship 1995-10-06 FOB
    line 2  part 29996   supp 9997  qty  31  ext   59705.69  disc 0.06  tax 0.07  NO  ship 1995-08-11 SHIP
    line 3  part 130760  supp 761   qty  49  ext   87747.24  disc 0.05  tax 0.05  NO  ship 1995-10-09 SHIP
    line 4  part 74888   supp 4889  qty   4  ext    7451.52  disc 0.03  tax 0.01  NO  ship 1995-11-01 MAIL
    line 5  part 29678   supp 9679  qty  33  ext   53053.11  disc 0.03  tax 0.03  NO  ship 1995-08-10 RAIL

ORDER 4240038  1996-06-19  status O  3-MEDIUM  clerk Clerk#000000055  total 124670.76
    line 1  part 140367  supp 2882  qty  34  ext   47850.24  disc 0.02  tax 0.03  NO  ship 1996-06-21 TRUCK
    line 2  part 131694  supp 1695  qty  44  ext   75930.36  disc 0.06  tax 0.07  NO  ship 1996-10-05 REG AIR

ORDER 464128  1997-09-02  status O  1-URGENT  clerk Clerk#000000604  total 215871.48
    line 1  part 130459  supp 5486  qty   3  ext    4468.35  disc 0.08  tax 0.06  NO  ship 1997-09-17 RAIL
    line 2  part 9677    supp 7178  qty  23  ext   36493.41  disc 0.00  tax 0.02  NO  ship 1997-10-16 SHIP
    line 3  part 36268   supp 3778  qty  35  ext   42149.10  disc 0.01  tax 0.06  NO  ship 1997-09-13 MAIL
    line 4  part 141244  supp 6273  qty  47  ext   60406.28  disc 0.05  tax 0.04  NO  ship 1997-12-29 SHIP
    line 5  part 84867   supp 9884  qty  28  ext   51852.08  disc 0.03  tax 0.02  NO  ship 1997-10-02 SHIP
    line 6  part 149322  supp 1837  qty  14  ext   19198.48  disc 0.08  tax 0.08  NO  ship 1997-10-20 AIR
```

```
ORDER 5783393  1997-12-31  status O  4-NOT SPECIFIED  clerk Clerk#000000779  total 93382.65
    line 1  part 72936   supp 7951  qty  30  ext   57267.90  disc 0.08  tax 0.05  NO  ship 1998-03-04 REG AIR
    line 2  part 22244   supp 7249  qty  12  ext   13994.88  disc 0.02  tax 0.02  NO  ship 1998-03-01 TRUCK
    line 3  part 51371   supp 8887  qty  14  ext   18513.18  disc 0.08  tax 0.08  NO  ship 1998-03-15 SHIP
    line 4  part 177906  supp 7907  qty   3  ext    5951.70  disc 0.10  tax 0.06  NO  ship 1998-02-06 SHIP

ORDER 1432258  1998-07-27  status O  1-URGENT  clerk Clerk#000000590  total 77070.43
    line 1  part 193242  supp 8281  qty  11  ext   14687.64  disc 0.10  tax 0.06  NO  ship 1998-11-06 SHIP
    line 2  part 5267    supp 268   qty  26  ext   30478.76  disc 0.00  tax 0.08  NO  ship 1998-11-18 TRUCK
    line 3  part 120825  supp 826   qty   5  ext    9229.10  disc 0.05  tax 0.02  NO  ship 1998-11-19 FOB
    line 4  part 91351   supp 3861  qty  16  ext   21477.60  disc 0.06  tax 0.05  NO  ship 1998-08-06 REG AIR

15 orders, 57 line items, lifetime order value 1835102.03
(1 GetItem + 1 Query page(s), 18.3 ms)
```

6.8 Q5: items ordered by customer 121 in 1994

```
Query  pk=CUST#121  sk begins_with ORDER#1994
orderdate  orderkey   line partkey  suppkey   qty     extprice
1994-01-24 3960641    1    162920   7953        2      3965.84
1994-01-24 3960641    2    188887   1406       38     75083.44
1994-04-09 2174404    1    146032   3575       13     14014.39
1994-04-09 2174404    2    148863   8864       15     28677.90
1994-04-09 2174404    3    138205   719        13     16161.60
1994-06-26 5512869    1    158243   5789        1      1301.24
1994-06-26 5512869    2    173959   3960        1      2032.95
1994-06-26 5512869    3    123950   8975       40     78958.00
1994-06-26 5512869    4    26304    6305        1      1230.30
1994-06-26 5512869    5    83046    5555       31     31900.24
1994-06-26 5512869    6    149742   9743        4      7166.96
1994-06-26 5512869    7    100340   5361       12     16084.08
1994-10-01 5299460    1    120498   3011       17     25814.33
1994-10-01 5299460    2    4077     9078        3      2943.21
1994-10-01 5299460    3    114001   1535       49     49735.00
1994-10-01 5299460    4    13391    8394       22     28696.58
1994-10-01 5299460    5    47432    9937       30     41382.90
1994-10-10 202628     1    7049     7050       22     21032.88
1994-10-10 202628     2    59993    5004       16     31247.84
1994-10-10 202628     3    92560    5070       20     31051.20
5 orders in 1994, 20 line items, 20 distinct parts: [4077, 7049, 13391, 26304, 47432, 59993, 83046, 92560, 100340, 114001, 120498, 123950, 138205, 146032, 14886
3, 149742, 158243, 162920, 173959, 188887]
(1 Query page(s), 11.7 ms)
```

6.9 Q6: items customer 121 ordered in 1993 but not in 1994

```
Query  pk=CUST#121  sk begins_with ORDER#1993
Query  pk=CUST#121  sk begins_with ORDER#1994
parts ordered in 1993 : 7 distinct [36749, 42471, 123613, 161673, 175816, 195405, 196580]
parts ordered in 1994 : 20 distinct [4077, 7049, 13391, 26304, 47432, 59993, 83046, 92560, 100340, 114001, 120498, 123950, 138205, 146032, 148863, 149742, 15824
3, 162920, 173959, 188887]
in 1993 but not in 1994: 7 parts
partkey  orderdate  orderkey   line   qty     extprice
175816   1993-06-24 3568994    1       49     92698.69
123613   1993-06-24 3568994    2        6      9819.66
161673   1993-06-24 3568994    3       34     58978.78
36749    1993-06-24 3568994    4       31     52257.94
42471    1993-10-16 2970276    1       47     66433.09
196580   1993-10-16 2970276    2       28     46944.24
195405   1993-12-26 2383682    1        2      3000.80
(2 Query page(s), 16.1 ms)
```

6.10 Q7: total value of the orders of customer 121 in 1994

```
Query  pk=CUST#121  sk begins_with ORDER#1994
orderdate  orderkey   status   totalprice
1994-01-24 3960641    F          77312.55
1994-04-09 2174404    F          55650.77
1994-06-26 5512869    F         133146.02
1994-10-01 5299460    F         147099.69
1994-10-10 202628     F          85006.64
5 orders in 1994, total value 498215.67
cross-check from line items, sum of extendedprice*(1-discount)*(1+tax) = 498215.88
(1 Query page(s), 13.3 ms)
```

7. Source code

All of `lab10/`: line-numbered and syntax-highlighted Java, XML and bash.

Lab10.java

`lab10/src/main/java/pmj/bdp/dynamo/lab10/Lab10.java`

```java
package pmj.bdp.dynamo.lab10;

import java.nio.file.Path;

import software.amazon.awssdk.enhanced.dynamodb.DynamoDbEnhancedClient;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbTable;
import software.amazon.awssdk.enhanced.dynamodb.TableSchema;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;

/**
 * Lab 10: TPC-H CUSTOMER / ORDERS / LINEITEM in DynamoDB (Enhanced Client).
 *
 * <pre>
 *   java ... Lab10 create [--drop]                create table TPCH + GSI OrderIndex
```

```
 *   java ... Lab10 load dataDir                  populate from customer.tbl, orders.tbl, lineitem.tbl
 *   java ... Lab10 describe                      describe-table
 *   java ... Lab10 q1 custkey                    customer profile
 *   java ... Lab10 q2 custkey                    all orders of a customer
 *   java ... Lab10 q3 orderkey                   all line items of an order
 *   java ... Lab10 q4 custkey                    order history (profile + orders + line items)
 *   java ... Lab10 q5 custkey year               items ordered in a year
 *   java ... Lab10 q6 custkey year               items ordered in year-1 but not in year
 *   java ... Lab10 q7 custkey year               total value of the orders of a year
 *   java ... Lab10 queries custkey orderkey year   q1 .. q7
 *   java ... Lab10 all dataDir custkey orderkey year
 * </pre>
 */
public class Lab10 {

    public static void main(String[] args) throws Exception {
        System.setProperty("org.slf4j.simpleLogger.defaultLogLevel", "warn");
        if (args.length < 1) {
            usage();
            return;
        }
        try (DynamoDbClient client = Dynamo.client()) {
            DynamoDbEnhancedClient enhanced = Dynamo.enhanced(client);
            DynamoDbTable<Customer> customers = enhanced.table(Keys.TABLE, TableSchema.fromBean(Customer.class));
            DynamoDbTable<Order> orders = enhanced.table(Keys.TABLE, TableSchema.fromBean(Order.class));
            Queries q = new Queries(enhanced);
            switch (args[0]) {
                case "create" -> Schema.create(client, orders, args.length > 1 && args[1].equals("--drop"));
                case "describe" -> describe();
                case "load" -> TpchLoader.load(enhanced, customers, orders, Path.of(args[1])).print();
                case "q1" -> q.q1CustomerProfile(Long.parseLong(args[1]));
                case "q2" -> q.q2OrdersOfCustomer(Long.parseLong(args[1]));
                case "q3" -> q.q3LineItemsOfOrder(Long.parseLong(args[1]));
                case "q4" -> q.q4OrderHistory(Long.parseLong(args[1]));
                case "q5" -> q.q5ItemsOrderedInYear(Long.parseLong(args[1]), Integer.parseInt(args[2]));
                case "q6" -> q.q6ItemsOfPreviousYearNotThisYear(Long.parseLong(args[1]), Integer.parseInt(args[2]));
                case "q7" -> q.q7TotalValueInYear(Long.parseLong(args[1]), Integer.parseInt(args[2]));
                case "queries" -> queries(q, Long.parseLong(args[1]), Long.parseLong(args[2]), Integer.parseInt(args[3]));
                case "all" -> {
                    Schema.create(client, orders, true);
                    TpchLoader.load(enhanced, customers, orders, Path.of(args[1])).print();
                    describe();
                    queries(q, Long.parseLong(args[2]), Long.parseLong(args[3]), Integer.parseInt(args[4]));
                }
                default -> usage();
            }
        }
    }

    /** describe-table with the long-timeout client (DynamoDB Local counts all items for it). */
    static void describe() {
        try (DynamoDbClient slow = Dynamo.longTimeoutClient()) {
            Schema.describe(slow);
        }
    }

    /** Runs the seven query functions for one customer, one of its orders and one year. */
    static void queries(Queries q, long custkey, long orderkey, int year) {
        q.q1CustomerProfile(custkey);
        q.q2OrdersOfCustomer(custkey);
        q.q3LineItemsOfOrder(orderkey);
        q.q4OrderHistory(custkey);
        q.q5ItemsOrderedInYear(custkey, year);
        q.q6ItemsOfPreviousYearNotThisYear(custkey, year);
        q.q7TotalValueInYear(custkey, year);
    }

    private static void usage() {
        System.out.println("usage: Lab10 create [--drop] | load dataDir | describe | q1..q7 ... | queries custkey orderkey year"
                + " | all dataDir custkey orderkey year");
    }
}
```

Queries.java

lab10/src/main/java/pmj/bdp/dynamo/lab10/Queries.java

```
package pmj.bdp.dynamo.lab10;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Set;
import java.util.TreeSet;

import software.amazon.awssdk.enhanced.dynamodb.DynamoDbEnhancedClient;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbIndex;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbTable;
import software.amazon.awssdk.enhanced.dynamodb.Key;
import software.amazon.awssdk.enhanced.dynamodb.TableSchema;
import software.amazon.awssdk.enhanced.dynamodb.model.Page;
import software.amazon.awssdk.enhanced.dynamodb.model.QueryConditional;

/**
 * The seven queries of the lab, one method each, all through the high-level (Enhanced) API:
```

```java
 * DynamoDbTable.getItem / query and DynamoDbIndex.query. No Scan anywhere.
 */
final class Queries {

    private final DynamoDbTable<Customer> customers;
    private final DynamoDbTable<Order> orders;
    private final DynamoDbIndex<Order> orderIndex;
    private int pages;                    // Query pages fetched by the last ordersOf(...) call

    Queries(DynamoDbEnhancedClient enhanced) {
        customers = enhanced.table(Keys.TABLE, TableSchema.fromBean(Customer.class));
        orders = enhanced.table(Keys.TABLE, TableSchema.fromBean(Order.class));
        orderIndex = orders.index(Keys.ORDER_INDEX);
    }

    // ---------------------------------------------------------------- Q1

    /** Q1. Retrieve a customer's profile: one GetItem on (CUST#custkey, PROFILE). */
    Customer q1CustomerProfile(long custkey) {
        section("Q1: retrieve the profile of customer " + custkey);
        String pk = Keys.customerPk(custkey);
        System.out.println("GetItem  pk=" + pk + "  sk=" + Keys.PROFILE_SK);
        long t = System.nanoTime();
        Customer c = customers.getItem(Key.builder().partitionValue(pk).sortValue(Keys.PROFILE_SK).build());
        if (c == null) {
            System.out.println("customer not found");
        } else {
            printCustomer(c);
        }
        done(t, "1 GetItem");
        return c;
    }

    // ---------------------------------------------------------------- Q2

    /** Q2. All orders of a customer: one Query on the customer's partition, sk begins_with ORDER#. */
    List<Order> q2OrdersOfCustomer(long custkey) {
        section("Q2: all orders of customer " + custkey);
        System.out.println("Query  pk=" + Keys.customerPk(custkey) + "  sk begins_with " + Keys.ORDER_PREFIX);
        long t = System.nanoTime();
        List<Order> list = ordersOf(custkey, Keys.ORDER_PREFIX);
        printOrderHeader();
        for (Order o : list) {
            printOrderRow(o);
        }
        System.out.printf(Locale.ROOT, "%d orders%n", list.size());
        done(t, pages + " Query page(s)");
        return list;
    }

    // ---------------------------------------------------------------- Q3

    /**
     * Q3. All line items of an order, given only O_ORDERKEY: Query the GSI OrderIndex
     * (KEYS_ONLY, so it returns the order's pk/sk), then GetItem the order, whose item
     * embeds every line item.
     */
    Order q3LineItemsOfOrder(long orderkey) {
        section("Q3: all line items of order " + orderkey);
        System.out.println("Query  index " + Keys.ORDER_INDEX + "  orderkey=" + orderkey);
        long t = System.nanoTime();
        Order ref = null;
        int indexPages = 0;
        for (Page<Order> page : orderIndex.query(
                QueryConditional.keyEqualTo(Key.builder().partitionValue(orderkey).build()))) {
            indexPages++;
            for (Order o : page.items()) {
                ref = o;
            }
        }
        if (ref == null) {
            System.out.println("order not found");
            done(t, indexPages + " index Query page(s)");
            return null;
        }
        System.out.println("  -> pk=" + ref.getPk() + "  sk=" + ref.getSk());
        System.out.println("GetItem  pk=" + ref.getPk() + "  sk=" + ref.getSk());
        Order full = orders.getItem(Key.builder().partitionValue(ref.getPk()).sortValue(ref.getSk()).build());
        printOrderHeader();
        printOrderRow(full);
        printLineHeader();
        for (LineItem l : full.getLineitems()) {
            printLineRow(l);
        }
        System.out.printf(Locale.ROOT, "%d line items%n", full.getLineitems().size());
        done(t, indexPages + " index Query page(s) + 1 GetItem");
        return full;
    }

    // ---------------------------------------------------------------- Q4

    /**
     * Q4. Everything needed to display a customer's order history: the profile (GetItem)
     * and every order with its embedded line items (one Query on the same partition).
     */
    void q4OrderHistory(long custkey) {
        section("Q4: order history of customer " + custkey + " (profile + each order with its line items)");
```

```
        String pk = Keys.customerPk(custkey);
        System.out.println("GetItem  pk=" + pk + "  sk=" + Keys.PROFILE_SK);
        System.out.println("Query    pk=" + pk + "  sk begins_with " + Keys.ORDER_PREFIX);
        long t = System.nanoTime();
        Customer c = customers.getItem(Key.builder().partitionValue(pk).sortValue(Keys.PROFILE_SK).build());
        List<Order> list = ordersOf(custkey, Keys.ORDER_PREFIX);
        if (c != null) {
            System.out.printf(Locale.ROOT, "%s  (custkey %d, %s, nation %d, phone %s, balance %s)%n",
                    c.getName(), c.getCustkey(), c.getMktsegment(), c.getNationkey(), c.getPhone(), c.getAcctbal());
        }
        long lines = 0;
        BigDecimal total = BigDecimal.ZERO;
        for (Order o : list) {
            System.out.printf(Locale.ROOT, "%nORDER %d  %s  status %s  %s  clerk %s  total %s%n",
                    o.getOrderkey(), o.getOrderdate(), o.getOrderstatus(), o.getOrderpriority(),
                    o.getClerk(), o.getTotalprice());
            for (LineItem l : o.getLineitems()) {
                System.out.printf(Locale.ROOT,
                        "    line %d  part %-7d supp %-5d qty %3s  ext %10s  disc %s  tax %s  %s%s  ship %s %s%n",
                        l.getLinenumber(), l.getPartkey(), l.getSuppkey(), l.getQuantity(), l.getExtendedprice(),
                        l.getDiscount(), l.getTax(), l.getReturnflag(), l.getLinestatus(), l.getShipdate(),
                        l.getShipmode());
                lines++;
            }
            total = total.add(o.getTotalprice());
        }
        System.out.printf(Locale.ROOT, "%n%d orders, %d line items, lifetime order value %s%n",
                list.size(), lines, total);
        done(t, "1 GetItem + " + pages + " Query page(s)");
    }

    // ------------------------------------------------------------------ Q5

    /** Q5. All items ordered by a customer in a year: one Query, sk begins_with ORDER#&lt;year&gt;. */
    List<LineItem> q5ItemsOrderedInYear(long custkey, int year) {
        section("Q5: items ordered by customer " + custkey + " in " + year);
        System.out.println("Query  pk=" + Keys.customerPk(custkey) + "  sk begins_with " + Keys.yearPrefix(year));
        long t = System.nanoTime();
        List<Order> list = ordersOf(custkey, Keys.yearPrefix(year));
        List<LineItem> items = new ArrayList<>();
        Set<Long> parts = new TreeSet<>();
        System.out.printf(Locale.ROOT, "%-10s %-10s %-4s %-8s %-7s %5s %12s%n",
                "orderdate", "orderkey", "line", "partkey", "suppkey", "qty", "extprice");
        for (Order o : list) {
            for (LineItem l : o.getLineitems()) {
                System.out.printf(Locale.ROOT, "%-10s %-10d %-4d %-8d %-7d %5s %12s%n",
                        o.getOrderdate(), o.getOrderkey(), l.getLinenumber(), l.getPartkey(), l.getSuppkey(),
                        l.getQuantity(), l.getExtendedprice());
                items.add(l);
                parts.add(l.getPartkey());
            }
        }
        System.out.printf(Locale.ROOT, "%d orders in %d, %d line items, %d distinct parts: %s%n",
                list.size(), year, items.size(), parts.size(), parts);
        done(t, pages + " Query page(s)");
        return items;
    }

    // ------------------------------------------------------------------ Q6

    /**
     * Q6. Items the customer ordered in (year-1) but not in year: two Queries
     * (one per year prefix) and a set difference on the part keys.
     */
    Set<Long> q6ItemsOfPreviousYearNotThisYear(long custkey, int year) {
        int previous = year - 1;
        section("Q6: items customer " + custkey + " ordered in " + previous + " but not in " + year);
        String pk = Keys.customerPk(custkey);
        System.out.println("Query  pk=" + pk + "  sk begins_with " + Keys.yearPrefix(previous));
        System.out.println("Query  pk=" + pk + "  sk begins_with " + Keys.yearPrefix(year));
        long t = System.nanoTime();
        List<Order> prevOrders = ordersOf(custkey, Keys.yearPrefix(previous));
        int prevPages = pages;
        List<Order> thisOrders = ordersOf(custkey, Keys.yearPrefix(year));
        Set<Long> prevParts = partsOf(prevOrders);
        Set<Long> thisParts = partsOf(thisOrders);
        Set<Long> onlyPrevious = new TreeSet<>(prevParts);
        onlyPrevious.removeAll(thisParts);
        System.out.printf(Locale.ROOT, "parts ordered in %d : %d distinct %s%n", previous, prevParts.size(), prevParts);
        System.out.printf(Locale.ROOT, "parts ordered in %d : %d distinct %s%n", year, thisParts.size(), thisParts);
        System.out.printf(Locale.ROOT, "in %d but not in %d: %d parts%n", previous, year, onlyPrevious.size());
        System.out.printf(Locale.ROOT, "%-8s %-10s %-10s %-4s %5s %12s%n",
                "partkey", "orderdate", "orderkey", "line", "qty", "extprice");
        for (Order o : prevOrders) {
            for (LineItem l : o.getLineitems()) {
                if (onlyPrevious.contains(l.getPartkey())) {
                    System.out.printf(Locale.ROOT, "%-8d %-10s %-10d %-4d %5s %12s%n",
                            l.getPartkey(), o.getOrderdate(), o.getOrderkey(), l.getLinenumber(),
                            l.getQuantity(), l.getExtendedprice());
                }
            }
        }
        done(t, (prevPages + pages) + " Query page(s)");
        return onlyPrevious;
    }

    // ------------------------------------------------------------------ Q7
```

```java

    /** Q7. Total value of a customer's orders in a given year: one Query, sum of O_TOTALPRICE. */
    BigDecimal q7TotalValueInYear(long custkey, int year) {
        section("Q7: total value of the orders of customer " + custkey + " in " + year);
        System.out.println("Query  pk=" + Keys.customerPk(custkey) + "  sk begins_with " + Keys.yearPrefix(year));
        long t = System.nanoTime();
        List<Order> list = ordersOf(custkey, Keys.yearPrefix(year));
        BigDecimal total = BigDecimal.ZERO;
        BigDecimal fromLines = BigDecimal.ZERO;
        System.out.printf(Locale.ROOT, "%-10s %-10s %-6s %12s%n", "orderdate", "orderkey", "status", "totalprice");
        for (Order o : list) {
            System.out.printf(Locale.ROOT, "%-10s %-10d %-6s %12s%n",
                    o.getOrderdate(), o.getOrderkey(), o.getOrderstatus(), o.getTotalprice());
            total = total.add(o.getTotalprice());
            for (LineItem l : o.getLineitems()) {
                fromLines = fromLines.add(l.netValue());
            }
        }
        System.out.printf(Locale.ROOT, "%d orders in %d, total value %s%n", list.size(), year, total);
        System.out.printf(Locale.ROOT, "cross-check from line items, sum of extendedprice*(1-discount)*(1+tax) = %s%n",
                fromLines.setScale(2, java.math.RoundingMode.HALF_UP));
        done(t, pages + " Query page(s)");
        return total;
    }

    // ------------------------------------------------------------------ helpers

    /** All orders of a customer whose sort key begins with the prefix, oldest first. */
    private List<Order> ordersOf(long custkey, String skPrefix) {
        QueryConditional cond = QueryConditional.sortBeginsWith(
                Key.builder().partitionValue(Keys.customerPk(custkey)).sortValue(skPrefix).build());
        List<Order> out = new ArrayList<>();
        pages = 0;
        for (Page<Order> page : orders.query(cond)) {
            pages++;
            out.addAll(page.items());
        }
        return out;
    }

    private static Set<Long> partsOf(List<Order> list) {
        Set<Long> parts = new TreeSet<>();
        for (Order o : list) {
            for (LineItem l : o.getLineitems()) {
                parts.add(l.getPartkey());
            }
        }
        return parts;
    }

    private static void printCustomer(Customer c) {
        System.out.println("custkey    : " + c.getCustkey());
        System.out.println("name       : " + c.getName());
        System.out.println("address    : " + c.getAddress());
        System.out.println("nationkey  : " + c.getNationkey());
        System.out.println("phone      : " + c.getPhone());
        System.out.println("acctbal    : " + c.getAcctbal());
        System.out.println("mktsegment : " + c.getMktsegment());
        System.out.println("comment    : " + c.getComment());
    }

    private static void printOrderHeader() {
        System.out.printf(Locale.ROOT, "%-10s %-10s %-6s %-15s %-15s %12s %5s%n",
                "orderdate", "orderkey", "status", "priority", "clerk", "totalprice", "lines");
    }

    private static void printOrderRow(Order o) {
        System.out.printf(Locale.ROOT, "%-10s %-10d %-6s %-15s %-15s %12s %5d%n",
                o.getOrderdate(), o.getOrderkey(), o.getOrderstatus(), o.getOrderpriority(), o.getClerk(),
                o.getTotalprice(), o.getLineitems() == null ? 0 : o.getLineitems().size());
    }

    private static void printLineHeader() {
        System.out.printf(Locale.ROOT, "%-4s %-8s %-7s %5s %12s %5s %5s %-2s %-2s %-10s %-10s %-10s %-17s %-7s%n",
                "line", "partkey", "suppkey", "qty", "extprice", "disc", "tax", "rf", "ls",
                "shipdate", "commitdate", "receipt", "shipinstruct", "mode");
    }

    private static void printLineRow(LineItem l) {
        System.out.printf(Locale.ROOT, "%-4d %-8d %-7d %5s %12s %5s %5s %-2s %-2s %-10s %-10s %-10s %-17s %-7s%n",
                l.getLinenumber(), l.getPartkey(), l.getSuppkey(), l.getQuantity(), l.getExtendedprice(),
                l.getDiscount(), l.getTax(), l.getReturnflag(), l.getLinestatus(), l.getShipdate(),
                l.getCommitdate(), l.getReceiptdate(), l.getShipinstruct(), l.getShipmode());
    }

    private static void section(String title) {
        System.out.println();
        System.out.println("======== " + title + " ========");
    }

    private static void done(long startNanos, String requests) {
        System.out.printf(Locale.ROOT, "(%s, %.1f ms)%n", requests, (System.nanoTime() - startNanos) / 1e6);
    }
}
```

TpchLoader.java

lab10/src/main/java/pmj/bdp/dynamo/lab10/TpchLoader.java

```java
package pmj.bdp.dynamo.lab10;

import java.io.BufferedReader;
import java.io.IOException;
import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.atomic.AtomicReference;
import java.util.stream.Stream;

import software.amazon.awssdk.enhanced.dynamodb.DynamoDbEnhancedClient;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbTable;
import software.amazon.awssdk.enhanced.dynamodb.model.BatchWriteItemEnhancedRequest;
import software.amazon.awssdk.enhanced.dynamodb.model.BatchWriteResult;
import software.amazon.awssdk.enhanced.dynamodb.model.WriteBatch;

/**
 * Populates the TPCH table from customer.tbl, orders.tbl and lineitem.tbl.
 *
 * customer.tbl rows become profile items. orders.tbl and lineitem.tbl are both written by dbgen
 * in ascending O_ORDERKEY order, so they are merge-joined in one streaming pass: each order is
 * read, its line items are taken from the head of lineitem.tbl, and one order item (with the
 * line items embedded) is produced. Items are written with BatchWriteItem (25 per request)
 * through the Enhanced Client, several requests in flight at a time.
 */
final class TpchLoader {

    static final int BATCH_SIZE = 25;                       // BatchWriteItem limit
    static final int THREADS = Integer.getInteger("ddb.threads", 8);
    static final int CUSTOMER_FIELDS = 8;
    static final int ORDER_FIELDS = 9;
    static final int LINE_FIELDS = 16;

    private TpchLoader() {
    }

    record Report(Path customerTbl, Path ordersTbl, Path lineTbl,
                  long customers, long orders, long lineItems,
                  long customerItems, long orderItems, long retries,
                  double customerSeconds, double orderSeconds) {

        void print() {
            System.out.println();
            System.out.println("======== load summary ========");
            System.out.printf(Locale.ROOT, "customer.tbl rows   : %,d%n", customers);
            System.out.printf(Locale.ROOT, "orders.tbl rows     : %,d%n", orders);
            System.out.printf(Locale.ROOT, "lineitem.tbl rows   : %,d (embedded in their orders)%n", lineItems);
            System.out.printf(Locale.ROOT, "profile items put   : %,d in %.1f s (%,.0f items/s)%n",
                    customerItems, customerSeconds, customerItems / customerSeconds);
            System.out.printf(Locale.ROOT, "order items put     : %,d in %.1f s (%,.0f items/s)%n",
                    orderItems, orderSeconds, orderItems / orderSeconds);
            System.out.printf(Locale.ROOT, "items in table      : %,d%n", customerItems + orderItems);
            System.out.printf(Locale.ROOT, "batch retries       : %,d (unprocessed items re-sent)%n", retries);
            System.out.printf(Locale.ROOT, "writer threads      : %d, %d items per BatchWriteItem%n", THREADS, BATCH_SIZE);
        }
    }

    static Report load(DynamoDbEnhancedClient enhanced, DynamoDbTable<Customer> customerTable,
                       DynamoDbTable<Order> orderTable, Path dataDir) throws Exception {
        Path customerTbl = findTable(dataDir, "customer");
        Path ordersTbl = findTable(dataDir, "orders");
        Path lineTbl = findTable(dataDir, "lineitem");
        System.out.println("customer.tbl = " + customerTbl + " (" + Files.size(customerTbl) + " bytes)");
        System.out.println("orders.tbl   = " + ordersTbl + " (" + Files.size(ordersTbl) + " bytes)");
        System.out.println("lineitem.tbl = " + lineTbl + " (" + Files.size(lineTbl) + " bytes)");

        System.out.println();
        System.out.println("---- phase 1: CUSTOMER -> profile items (pk=CUST#custkey, sk=PROFILE) ----");
        long t0 = System.nanoTime();
        BatchWriter<Customer> cw = new BatchWriter<>(enhanced, customerTable, Customer.class, "CUSTOMER");
        long customers = loadCustomers(customerTbl, cw);
        cw.finish();
        double customerSeconds = (System.nanoTime() - t0) / 1e9;

        System.out.println();
        System.out.println("---- phase 2: ORDERS + LINEITEM -> order items (pk=CUST#custkey, sk=ORDER#date#orderkey) ----");
        long t1 = System.nanoTime();
        BatchWriter<Order> ow = new BatchWriter<>(enhanced, orderTable, Order.class, "ORDER");
        long[] counts = loadOrders(ordersTbl, lineTbl, ow);
        ow.finish();
        double orderSeconds = (System.nanoTime() - t1) / 1e9;

        return new Report(customerTbl, ordersTbl, lineTbl, customers, counts[0], counts[1],
                cw.written(), ow.written(), cw.retries() + ow.retries(), customerSeconds, orderSeconds);
```

```
    }

    private static long loadCustomers(Path tbl, BatchWriter<Customer> writer) throws Exception {
        long rows = 0;
        try (BufferedReader reader = Files.newBufferedReader(tbl, StandardCharsets.US_ASCII)) {
            String[] fields;
            while ((fields = nextFields(reader, CUSTOMER_FIELDS)) != null) {
                writer.add(parseCustomer(fields));
                rows++;
            }
        }
        return rows;
    }

    /** Merge-join of orders.tbl with lineitem.tbl; both files are ordered by O_ORDERKEY. */
    private static long[] loadOrders(Path ordersTbl, Path lineTbl, BatchWriter<Order> writer) throws Exception {
        long orders = 0;
        long lines = 0;
        try (BufferedReader orderReader = Files.newBufferedReader(ordersTbl, StandardCharsets.US_ASCII);
             BufferedReader lineReader = Files.newBufferedReader(lineTbl, StandardCharsets.US_ASCII)) {
            String[] pendingLine = nextFields(lineReader, LINE_FIELDS);
            String[] fields;
            while ((fields = nextFields(orderReader, ORDER_FIELDS)) != null) {
                Order order = parseOrder(fields);
                long orderkey = order.getOrderkey();
                List<LineItem> items = new ArrayList<>(7);
                while (pendingLine != null) {
                    long lineOrderkey = Long.parseLong(pendingLine[0]);
                    if (lineOrderkey < orderkey) {
                        throw new IllegalStateException("lineitem.tbl is not ordered by orderkey or has an orphan line: "
                                + "L_ORDERKEY " + lineOrderkey + " seen while at O_ORDERKEY " + orderkey);
                    }
                    if (lineOrderkey > orderkey) {
                        break;
                    }
                    items.add(parseLineItem(pendingLine));
                    lines++;
                    pendingLine = nextFields(lineReader, LINE_FIELDS);
                }
                order.setLineitems(items);
                writer.add(order);
                orders++;
            }
            if (pendingLine != null) {
                throw new IllegalStateException("line items left over after the last order, L_ORDERKEY " + pendingLine[0]);
            }
        }
        return new long[] {orders, lines};
    }

    static Customer parseCustomer(String[] f) {
        long custkey = Long.parseLong(f[0]);
        Customer c = new Customer();
        c.setPk(Keys.customerPk(custkey));
        c.setSk(Keys.PROFILE_SK);
        c.setType(Keys.TYPE_CUSTOMER);
        c.setCustkey(custkey);
        c.setName(f[1]);
        c.setAddress(f[2]);
        c.setNationkey(Integer.parseInt(f[3]));
        c.setPhone(f[4]);
        c.setAcctbal(new BigDecimal(f[5]));
        c.setMktsegment(f[6]);
        c.setComment(f[7]);
        return c;
    }

    static Order parseOrder(String[] f) {
        long orderkey = Long.parseLong(f[0]);
        long custkey = Long.parseLong(f[1]);
        String orderdate = f[4];
        Order o = new Order();
        o.setPk(Keys.customerPk(custkey));
        o.setSk(Keys.orderSk(orderdate, orderkey));
        o.setType(Keys.TYPE_ORDER);
        o.setOrderkey(orderkey);
        o.setCustkey(custkey);
        o.setOrderstatus(f[2]);
        o.setTotalprice(new BigDecimal(f[3]));
        o.setOrderdate(orderdate);
        o.setYear(Integer.parseInt(orderdate.substring(0, 4)));
        o.setOrderpriority(f[5]);
        o.setClerk(f[6]);
        o.setShippriority(Integer.parseInt(f[7]));
        o.setComment(f[8]);
        return o;
    }

    static LineItem parseLineItem(String[] f) {
        LineItem l = new LineItem();
        l.setPartkey(Long.parseLong(f[1]));
        l.setSuppkey(Long.parseLong(f[2]));
        l.setLinenumber(Integer.parseInt(f[3]));
        l.setQuantity(new BigDecimal(f[4]));
        l.setExtendedprice(new BigDecimal(f[5]));
        l.setDiscount(new BigDecimal(f[6]));
        l.setTax(new BigDecimal(f[7]));
```

```
        l.setReturnflag(f[8]);
        l.setLinestatus(f[9]);
        l.setShipdate(f[10]);
        l.setCommitdate(f[11]);
        l.setReceiptdate(f[12]);
        l.setShipinstruct(f[13]);
        l.setShipmode(f[14]);
        l.setComment(f[15]);
        return l;
    }

    /** Next non-empty row of a .tbl file split into exactly n fields. */
    private static String[] nextFields(BufferedReader reader, int n) throws IOException {
        String line;
        while ((line = reader.readLine()) != null) {
            if (line.isEmpty()) {
                continue;
            }
            String[] fields = split(line, n);
            if (fields == null) {
                throw new IOException("malformed row (expected " + n + " fields): " + line);
            }
            return fields;
        }
        return null;
    }

    /** TPC-H .tbl rows are pipe-delimited and end with a trailing pipe. */
    static String[] split(String line, int n) {
        String[] out = new String[n];
        int start = 0;
        int found = 0;
        for (int i = 0; i < line.length(); i++) {
            if (line.charAt(i) == '|') {
                if (found == n) {
                    return null;
                }
                out[found++] = line.substring(start, i);
                start = i + 1;
            }
        }
        if (start < line.length()) {            // no trailing pipe
            if (found == n) {
                return null;
            }
            out[found++] = line.substring(start);
        }
        return found == n ? out : null;
    }

    private static Path findTable(Path dataDir, String baseName) throws IOException {
        try (Stream<Path> files = Files.walk(dataDir, 2)) {
            return files.filter(Files::isRegularFile)
                    .filter(p -> p.getFileName().toString().equalsIgnoreCase(baseName + ".tbl"))
                    .findFirst()
                    .orElseThrow(() -> new IOException("could not find " + baseName + ".tbl under " + dataDir));
        }
    }

    /** Groups items into batches of 25 and writes them with BatchWriteItem on a thread pool. */
    static final class BatchWriter<T> {

        private final DynamoDbEnhancedClient enhanced;
        private final DynamoDbTable<T> table;
        private final Class<T> type;
        private final String label;
        private final ExecutorService pool = Executors.newFixedThreadPool(THREADS);
        private final Semaphore inFlight = new Semaphore(THREADS * 4);
        private final AtomicLong written = new AtomicLong();
        private final AtomicLong retries = new AtomicLong();
        private final AtomicReference<Throwable> failure = new AtomicReference<>();
        private final long started = System.nanoTime();
        private List<T> batch = new ArrayList<>(BATCH_SIZE);
        private long submitted;

        BatchWriter(DynamoDbEnhancedClient enhanced, DynamoDbTable<T> table, Class<T> type, String label) {
            this.enhanced = enhanced;
            this.table = table;
            this.type = type;
            this.label = label;
        }

        void add(T item) throws InterruptedException {
            batch.add(item);
            if (batch.size() == BATCH_SIZE) {
                flushBatch();
            }
        }

        private void flushBatch() throws InterruptedException {
            if (batch.isEmpty()) {
                return;
            }
            Throwable f = failure.get();
            if (f != null) {
                throw new IllegalStateException(label + " batch write failed", f);
            }
```

```
            List<T> items = batch;
            batch = new ArrayList<>(BATCH_SIZE);
            inFlight.acquire();
            pool.execute(() -> {
                try {
                    write(items);
                } catch (Throwable t) {
                    failure.compareAndSet(null, t);
                } finally {
                    inFlight.release();
                }
            });
            submitted += items.size();
            if (submitted % 100_000 == 0) {
                progress();
            }
        }

        private void write(List<T> items) throws InterruptedException {
            List<T> pending = items;
            int attempt = 0;
            while (!pending.isEmpty()) {
                WriteBatch.Builder<T> wb = WriteBatch.builder(type).mappedTableResource(table);
                for (T item : pending) {
                    wb.addPutItem(item);
                }
                BatchWriteResult result = enhanced.batchWriteItem(
                        BatchWriteItemEnhancedRequest.builder().writeBatches(wb.build()).build());
                List<T> unprocessed = result.unprocessedPutItemsForTable(table);
                written.addAndGet(pending.size() - unprocessed.size());
                if (!unprocessed.isEmpty()) {
                    retries.incrementAndGet();
                    Thread.sleep(Math.min(1000L, 50L << Math.min(attempt++, 4)));
                }
                pending = unprocessed;
            }
        }

        void finish() throws Exception {
            flushBatch();
            pool.shutdown();
            pool.awaitTermination(1, TimeUnit.DAYS);
            Throwable f = failure.get();
            if (f != null) {
                throw new IllegalStateException(label + " batch write failed", f);
            }
            progress();
        }

        long written() {
            return written.get();
        }

        long retries() {
            return retries.get();
        }

        private void progress() {
            double sec = (System.nanoTime() - started) / 1e9;
            System.out.printf(Locale.ROOT, "  %-8s submitted %,10d  written %,10d  %7.1f s  %,7.0f items/s%n",
                    label, submitted, written.get(), sec, written.get() / Math.max(sec, 1e-9));
        }
    }
}
```

Schema.java

lab10/src/main/java/pmj/bdp/dynamo/lab10/Schema.java

```
package pmj.bdp.dynamo.lab10;

import java.util.Locale;

import software.amazon.awssdk.enhanced.dynamodb.DynamoDbTable;
import software.amazon.awssdk.enhanced.dynamodb.model.CreateTableEnhancedRequest;
import software.amazon.awssdk.enhanced.dynamodb.model.EnhancedGlobalSecondaryIndex;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeDefinition;
import software.amazon.awssdk.services.dynamodb.model.GlobalSecondaryIndexDescription;
import software.amazon.awssdk.services.dynamodb.model.KeySchemaElement;
import software.amazon.awssdk.services.dynamodb.model.Projection;
import software.amazon.awssdk.services.dynamodb.model.ProjectionType;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;
import software.amazon.awssdk.services.dynamodb.model.TableDescription;

/** Creates and describes the single table TPCH with its global secondary index OrderIndex. */
final class Schema {

    private Schema() {
    }

    static void create(DynamoDbClient client, DynamoDbTable<Order> orderTable, boolean drop) {
        if (exists(client)) {
            if (!drop) {
                System.out.println("table " + Keys.TABLE + " already exists (use --drop to recreate)");
                describe(client);
```

```
                return;
            }
            System.out.println("deleting existing table " + Keys.TABLE + " ...");
            client.deleteTable(r -> r.tableName(Keys.TABLE));
            client.waiter().waitUntilTableNotExists(r -> r.tableName(Keys.TABLE));
        }
        System.out.println("creating table " + Keys.TABLE + " (pk S, sk S) with GSI " + Keys.ORDER_INDEX
                + " (orderkey N, KEYS_ONLY), on-demand capacity ...");
        // The Order bean carries the key schema (pk, sk) and the GSI partition key (orderkey).
        orderTable.createTable(CreateTableEnhancedRequest.builder()
                .globalSecondaryIndices(EnhancedGlobalSecondaryIndex.builder()
                        .indexName(Keys.ORDER_INDEX)
                        .projection(Projection.builder().projectionType(ProjectionType.KEYS_ONLY).build())
                        .build())
                .build());
        client.waiter().waitUntilTableExists(r -> r.tableName(Keys.TABLE));
        System.out.println("created.");
        describe(client);
    }

    static boolean exists(DynamoDbClient client) {
        try {
            client.describeTable(r -> r.tableName(Keys.TABLE));
            return true;
        } catch (ResourceNotFoundException e) {
            return false;
        }
    }

    static void describe(DynamoDbClient client) {
        TableDescription t = client.describeTable(r -> r.tableName(Keys.TABLE)).table();
        System.out.println();
        System.out.println("======== describe-table " + t.tableName() + " ========");
        System.out.println("status               : " + t.tableStatusAsString());
        System.out.println("billing              : " + (t.billingModeSummary() == null ? "PROVISIONED"
                : t.billingModeSummary().billingModeAsString()));
        System.out.println("key schema           : " + keySchema(t.keySchema()));
        StringBuilder attrs = new StringBuilder();
        for (AttributeDefinition a : t.attributeDefinitions()) {
            attrs.append(a.attributeName()).append(' ').append(a.attributeTypeAsString()).append("  ");
        }
        System.out.println("attribute definitions: " + attrs.toString().trim());
        for (GlobalSecondaryIndexDescription g : t.globalSecondaryIndexes()) {
            System.out.println("GSI " + g.indexName() + "        : " + keySchema(g.keySchema())
                    + ", projection " + g.projection().projectionTypeAsString()
                    + ", status " + g.indexStatusAsString());
        }
        System.out.printf(Locale.ROOT, "item count          : %,d%n", t.itemCount());
        System.out.printf(Locale.ROOT, "table size          : %,d bytes (%.1f MiB)%n",
                t.tableSizeBytes(), t.tableSizeBytes() / (1024.0 * 1024.0));
    }

    private static String keySchema(java.util.List<KeySchemaElement> schema) {
        StringBuilder sb = new StringBuilder();
        for (KeySchemaElement k : schema) {
            if (sb.length() > 0) {
                sb.append(", ");
            }
            sb.append(k.attributeName()).append(" (").append(k.keyTypeAsString()).append(')');
        }
        return sb.toString();
    }
}
```

Keys.java

`lab10/src/main/java/pmj/bdp/dynamo/lab10/Keys.java`

```
package pmj.bdp.dynamo.lab10;

import java.util.Locale;

/**
 * Key layout of the single table {@code TPCH}.
 *
 * <pre>
 *   item             pk              sk                              GSI OrderIndex (orderkey)
 *   customer profile  CUST#&lt;custkey&gt;   PROFILE                         -
 *   order + lines     CUST#&lt;custkey&gt;   ORDER#&lt;orderdate&gt;#&lt;orderkey10&gt;    orderkey
 * </pre>
 *
 * Every order of a customer lives in the customer's partition, sorted by order date,
 * so "orders of a customer", "orders of a customer in a year" and "order history"
 * are one Query each. The line items of an order are embedded in the order item.
 */
final class Keys {

    static final String TABLE = "TPCH";
    static final String ORDER_INDEX = "OrderIndex";

    static final String TYPE_CUSTOMER = "CUSTOMER";
    static final String TYPE_ORDER = "ORDER";

    static final String PROFILE_SK = "PROFILE";
    static final String ORDER_PREFIX = "ORDER#";

```

```
    private Keys() {
    }

    static String customerPk(long custkey) {
        return "CUST#" + custkey;
    }

    /** ORDER#1996-01-02#0000000001 : date first so the sort key orders by date, key last for uniqueness. */
    static String orderSk(String orderdate, long orderkey) {
        return String.format(Locale.ROOT, "%s%s#%010d", ORDER_PREFIX, orderdate, orderkey);
    }

    /** Prefix shared by all orders of one year: ORDER#1996 */
    static String yearPrefix(int year) {
        return ORDER_PREFIX + year;
    }
}
```

Customer.java

lab10/src/main/java/pmj/bdp/dynamo/lab10/Customer.java

```
package pmj.bdp.dynamo.lab10;

import java.math.BigDecimal;

import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbBean;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbPartitionKey;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSortKey;

/** Customer profile item (TPC-H CUSTOMER row). pk = CUST#custkey, sk = PROFILE. */
@DynamoDbBean
public class Customer {

    private String pk;
    private String sk;
    private String type;
    private Long custkey;
    private String name;
    private String address;
    private Integer nationkey;
    private String phone;
    private BigDecimal acctbal;
    private String mktsegment;
    private String comment;

    public Customer() {
    }

    @DynamoDbPartitionKey
    public String getPk() {
        return pk;
    }

    public void setPk(String pk) {
        this.pk = pk;
    }

    @DynamoDbSortKey
    public String getSk() {
        return sk;
    }

    public void setSk(String sk) {
        this.sk = sk;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    public Long getCustkey() {
        return custkey;
    }

    public void setCustkey(Long custkey) {
        this.custkey = custkey;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
```

```
        this.address = address;
    }

    public Integer getNationkey() {
        return nationkey;
    }

    public void setNationkey(Integer nationkey) {
        this.nationkey = nationkey;
    }

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }

    public BigDecimal getAcctbal() {
        return acctbal;
    }

    public void setAcctbal(BigDecimal acctbal) {
        this.acctbal = acctbal;
    }

    public String getMktsegment() {
        return mktsegment;
    }

    public void setMktsegment(String mktsegment) {
        this.mktsegment = mktsegment;
    }

    public String getComment() {
        return comment;
    }

    public void setComment(String comment) {
        this.comment = comment;
    }
}
```

Order.java

lab10/src/main/java/pmj/bdp/dynamo/lab10/Order.java

```
package pmj.bdp.dynamo.lab10;

import java.math.BigDecimal;
import java.util.List;

import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbBean;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbPartitionKey;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondaryPartitionKey;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSortKey;

/**
 * Order item (TPC-H ORDERS row) with all of its LINEITEM rows embedded.
 * pk = CUST#custkey, sk = ORDER#orderdate#orderkey. orderkey is the partition key
 * of the global secondary index OrderIndex, so an order can also be found by O_ORDERKEY alone.
 */
@DynamoDbBean
public class Order {

    private String pk;
    private String sk;
    private String type;
    private Long orderkey;
    private Long custkey;
    private String orderstatus;
    private BigDecimal totalprice;
    private String orderdate;
    private Integer year;
    private String orderpriority;
    private String clerk;
    private Integer shippriority;
    private String comment;
    private List<LineItem> lineitems;

    public Order() {
    }

    @DynamoDbPartitionKey
    public String getPk() {
        return pk;
    }

    public void setPk(String pk) {
        this.pk = pk;
    }

    @DynamoDbSortKey
    public String getSk() {
        return sk;
```

```
    }

    public void setSk(String sk) {
        this.sk = sk;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    @DynamoDbSecondaryPartitionKey(indexNames = Keys.ORDER_INDEX)
    public Long getOrderkey() {
        return orderkey;
    }

    public void setOrderkey(Long orderkey) {
        this.orderkey = orderkey;
    }

    public Long getCustkey() {
        return custkey;
    }

    public void setCustkey(Long custkey) {
        this.custkey = custkey;
    }

    public String getOrderstatus() {
        return orderstatus;
    }

    public void setOrderstatus(String orderstatus) {
        this.orderstatus = orderstatus;
    }

    public BigDecimal getTotalprice() {
        return totalprice;
    }

    public void setTotalprice(BigDecimal totalprice) {
        this.totalprice = totalprice;
    }

    public String getOrderdate() {
        return orderdate;
    }

    public void setOrderdate(String orderdate) {
        this.orderdate = orderdate;
    }

    public Integer getYear() {
        return year;
    }

    public void setYear(Integer year) {
        this.year = year;
    }

    public String getOrderpriority() {
        return orderpriority;
    }

    public void setOrderpriority(String orderpriority) {
        this.orderpriority = orderpriority;
    }

    public String getClerk() {
        return clerk;
    }

    public void setClerk(String clerk) {
        this.clerk = clerk;
    }

    public Integer getShippriority() {
        return shippriority;
    }

    public void setShippriority(Integer shippriority) {
        this.shippriority = shippriority;
    }

    public String getComment() {
        return comment;
    }

    public void setComment(String comment) {
        this.comment = comment;
    }

    public List<LineItem> getLineitems() {
        return lineitems;
```

```
    }

    public void setLineitems(List<LineItem> lineitems) {
        this.lineitems = lineitems;
    }
}
```

LineItem.java

lab10/src/main/java/pmj/bdp/dynamo/lab10/LineItem.java

```
package pmj.bdp.dynamo.lab10;

import java.math.BigDecimal;

import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbBean;

/** One TPC-H LINEITEM row, stored as a map inside the order item's "lineitems" list. */
@DynamoDbBean
public class LineItem {

    private Integer linenumber;
    private Long partkey;
    private Long suppkey;
    private BigDecimal quantity;
    private BigDecimal extendedprice;
    private BigDecimal discount;
    private BigDecimal tax;
    private String returnflag;
    private String linestatus;
    private String shipdate;
    private String commitdate;
    private String receiptdate;
    private String shipinstruct;
    private String shipmode;
    private String comment;

    public LineItem() {
    }

    public Integer getLinenumber() {
        return linenumber;
    }

    public void setLinenumber(Integer linenumber) {
        this.linenumber = linenumber;
    }

    public Long getPartkey() {
        return partkey;
    }

    public void setPartkey(Long partkey) {
        this.partkey = partkey;
    }

    public Long getSuppkey() {
        return suppkey;
    }

    public void setSuppkey(Long suppkey) {
        this.suppkey = suppkey;
    }

    public BigDecimal getQuantity() {
        return quantity;
    }

    public void setQuantity(BigDecimal quantity) {
        this.quantity = quantity;
    }

    public BigDecimal getExtendedprice() {
        return extendedprice;
    }

    public void setExtendedprice(BigDecimal extendedprice) {
        this.extendedprice = extendedprice;
    }

    public BigDecimal getDiscount() {
        return discount;
    }

    public void setDiscount(BigDecimal discount) {
        this.discount = discount;
    }

    public BigDecimal getTax() {
        return tax;
    }

    public void setTax(BigDecimal tax) {
        this.tax = tax;
    }

```

```
    public String getReturnflag() {
        return returnflag;
    }

    public void setReturnflag(String returnflag) {
        this.returnflag = returnflag;
    }

    public String getLinestatus() {
        return linestatus;
    }

    public void setLinestatus(String linestatus) {
        this.linestatus = linestatus;
    }

    public String getShipdate() {
        return shipdate;
    }

    public void setShipdate(String shipdate) {
        this.shipdate = shipdate;
    }

    public String getCommitdate() {
        return commitdate;
    }

    public void setCommitdate(String commitdate) {
        this.commitdate = commitdate;
    }

    public String getReceiptdate() {
        return receiptdate;
    }

    public void setReceiptdate(String receiptdate) {
        this.receiptdate = receiptdate;
    }

    public String getShipinstruct() {
        return shipinstruct;
    }

    public void setShipinstruct(String shipinstruct) {
        this.shipinstruct = shipinstruct;
    }

    public String getShipmode() {
        return shipmode;
    }

    public void setShipmode(String shipmode) {
        this.shipmode = shipmode;
    }

    public String getComment() {
        return comment;
    }

    public void setComment(String comment) {
        this.comment = comment;
    }

    /** extendedprice * (1 - discount) * (1 + tax): what the customer pays for this line. */
    BigDecimal netValue() {
        return extendedprice.multiply(BigDecimal.ONE.subtract(discount))
                .multiply(BigDecimal.ONE.add(tax));
    }
}
```

Dynamo.java

lab10/src/main/java/pmj/bdp/dynamo/lab10/Dynamo.java

```
package pmj.bdp.dynamo.lab10;

import java.net.URI;
import java.time.Duration;

import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.awscore.retry.AwsRetryStrategy;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbEnhancedClient;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.DynamoDbClientBuilder;

/**
 * Connection to DynamoDB Local, same settings as the setup sheet
 * (endpoint http://localhost:8000, access key 1234 / secret 1234, region ap-south-1).
 */
final class Dynamo {

    static final String ENDPOINT = System.getProperty("ddb.endpoint", "http://localhost:8000");
```

```

    private Dynamo() {
    }

    /** Low-level client: talks to DynamoDB Local instead of AWS. SDK defaults (30 s socket timeout, retries). */
    static DynamoDbClient client() {
        return builder(Duration.ofSeconds(30)).build();
    }

    /**
     * DescribeTable on DynamoDB Local counts every row of its SQLite table, which takes minutes
     * on 1.65 million items, so that call gets a 30-minute timeout and no retries.
     */
    static DynamoDbClient longTimeoutClient() {
        Duration limit = Duration.ofMinutes(30);
        return builder(limit)
                .overrideConfiguration(o -> o.retryStrategy(AwsRetryStrategy.doNotRetry())
                        .apiCallTimeout(limit).apiCallAttemptTimeout(limit))
                .build();
    }

    private static DynamoDbClientBuilder builder(Duration socketTimeout) {
        return DynamoDbClient.builder()
                .endpointOverride(URI.create(ENDPOINT))
                .region(Region.AP_SOUTH_1)
                .credentialsProvider(StaticCredentialsProvider.create(
                        AwsBasicCredentials.create("1234", "1234")))
                .httpClientBuilder(ApacheHttpClient.builder()
                        .connectionTimeout(Duration.ofSeconds(10))
                        .socketTimeout(socketTimeout));
    }

    /** High-level (Enhanced) client built on top of the low-level one. */
    static DynamoDbEnhancedClient enhanced(DynamoDbClient client) {
        return DynamoDbEnhancedClient.builder().dynamoDbClient(client).build();
    }
}
```

pom.xml

lab10/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>pmj.bdp.dynamo</groupId>
    <artifactId>lab10</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>17</maven.compiler.release>
    </properties>

    <!-- Dependencies as given in the Lab 10 sheet -->
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>software.amazon.awssdk</groupId>
                <artifactId>bom</artifactId>
                <version>2.31.0</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <dependencies>
        <dependency>
            <groupId>software.amazon.awssdk</groupId>
            <artifactId>dynamodb</artifactId>
        </dependency>
        <dependency>
            <groupId>software.amazon.awssdk</groupId>
            <artifactId>dynamodb-enhanced</artifactId>
        </dependency>
        <dependency> <!-- default HTTP client of the SDK; declared so its timeouts can be set -->
            <groupId>software.amazon.awssdk</groupId>
            <artifactId>apache-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>1.7.36</version>
        </dependency>
    </dependencies>

    <build>
        <finalName>lab10</finalName>
        <plugins>
            <plugin>
```

```
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.5.0</version>
            </plugin>
        </plugins>
    </build>

</project>
```

run_all.sh

lab10/run_all.sh

```
#!/usr/bin/env bash
# IE-494 Lab 10: DynamoDB Local + TPC-H SF1 (CUSTOMER, ORDERS, LINEITEM).
#   bash run_all.sh [custkey] [orderkey] [year]
# Defaults: customer 121 (15 orders, 3 in 1993 and 5 in 1994), its order 5512869 (7 line items), year 1994.
set -euo pipefail
ROOT="$(cd "$(dirname "$0")" && pwd)"
cd "$ROOT"
DATA="${DATA:-$ROOT/../lab8/data}"          # customer.tbl, orders.tbl, lineitem.tbl (dbgen -s 1)
DBGEN="$ROOT/../lab8/tpch-kit/dbgen"
DDB="$ROOT/dynamodb_local"                  # unzipped dynamodb_local_latest
OUT="$ROOT/output"
CUST="${1:-121}"
ORDER="${2:-5512869}"
YEAR="${3:-1994}"
mkdir -p "$OUT" "$ROOT/db"

# 1. DynamoDB Local on :8000 (same command as the course setup sheet, plus a data directory)
if ! curl -s -o /dev/null http://localhost:8000/; then
  (cd "$DDB" && nohup java -Djava.library.path=./DynamoDBLocal_lib -jar DynamoDBLocal.jar \
      -sharedDb -port 8000 -dbPath "$ROOT/db" > "$OUT/dynamodb_local.log" 2>&1 &)
  sleep 4
fi

# 2. TPC-H SF1 tables, generated with the dbgen of Lab 08 if they are missing
if [ ! -f "$DATA/lineitem.tbl" ]; then
  mkdir -p "$DATA"
  (cd "$DBGEN" && DSS_PATH="$DATA" ./dbgen -q -s 1 -T c -f && DSS_PATH="$DATA" ./dbgen -q -s 1 -T o -f)
fi

# 3. build and run
mvn -q -DskipTests package
# exec:exec forks a plain JVM with the project classpath; under exec:java the SDK runs inside
# Maven's plugin classloader, which failed to load an SDK exception class once during this lab.
run() {
  mvn -q exec:exec -Dexec.executable=java \
      -Dexec.args="-cp %classpath pmj.bdp.dynamo.lab10.Lab10 $*"
}
run create --drop            2>&1 | tee "$OUT/01_create.txt"
run load "$DATA"             2>&1 | tee "$OUT/02_load.txt"
run describe                 2>&1 | tee "$OUT/03_describe.txt"
run queries "$CUST" "$ORDER" "$YEAR" 2>&1 | tee "$OUT/04_queries.txt"
cat "$OUT"/0[1-4]_*.txt > "$OUT/run_log.txt"
```